МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине

«Проектирование вычислительных систем»

Вариант: 5

Работу выполнили:
Студенты группы P3411
Болорболд Аригуун
Кравцов Кирилл Денисович

Преподаватель:
Пинкевич Василий Юрьевич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 4
   2.1. Используемые контакты 4
   2.2. Блок-схема: 5
   2.3. Код драйвера 5
3. Вывод 12

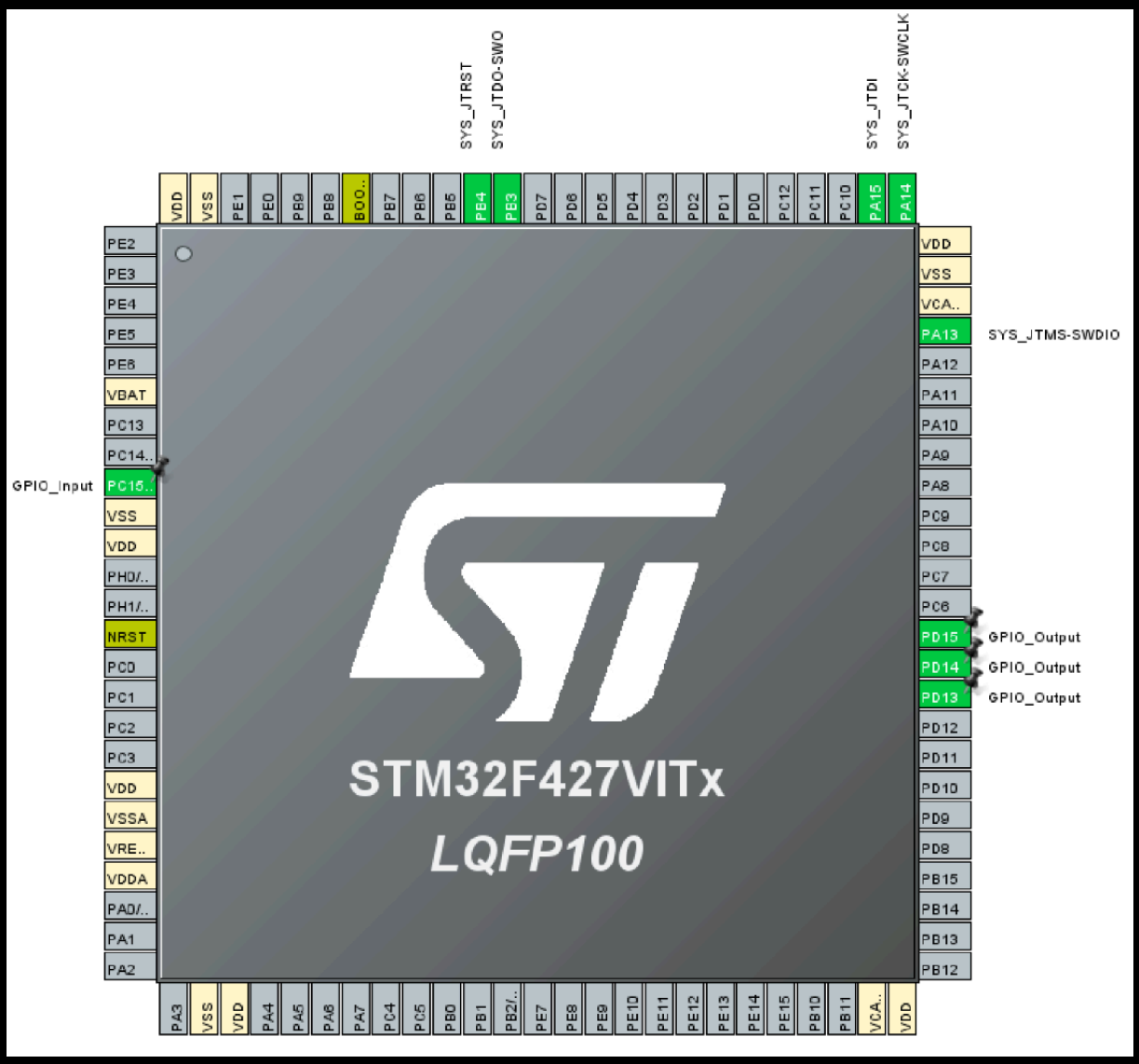
1. Текст задания

Разработать и реализовать драйверы управления светодиодными индикаторами и чтения состояния кнопки стенда SDK-1.1M (индикаторы и кнопка расположены на боковой панели стенда). Написать программу с использованием разработанных драйверов в соответствии с вариантом задания.

Реализовать «кодовый замок». После ввода единственно верной последовательности не менее чем из восьми коротких и длинных нажатий должен загореться зелёный светодиод, обозначающий «открытие» замка. Светодиод горит некоторое время, потом гаснет, и система вновь переходит в начало ввода. Правильно введённый элемент последовательности должен сопровождаться единичным миганием желтого светодиода. Неправильно введенный элемент – единичным миганием красного светодиода и сбросом в начало ввода. После трёх неправильных попыток ввода красный светодиод должен мигать не менее 5 секунд, и после этого происходит сброс в начало ввода. Если код начал вводиться, но за некоторое ограниченное время не был введен до конца, желтый светодиод выдает длинный импульс и происходит сброс в начало ввода.

2. Выполнение

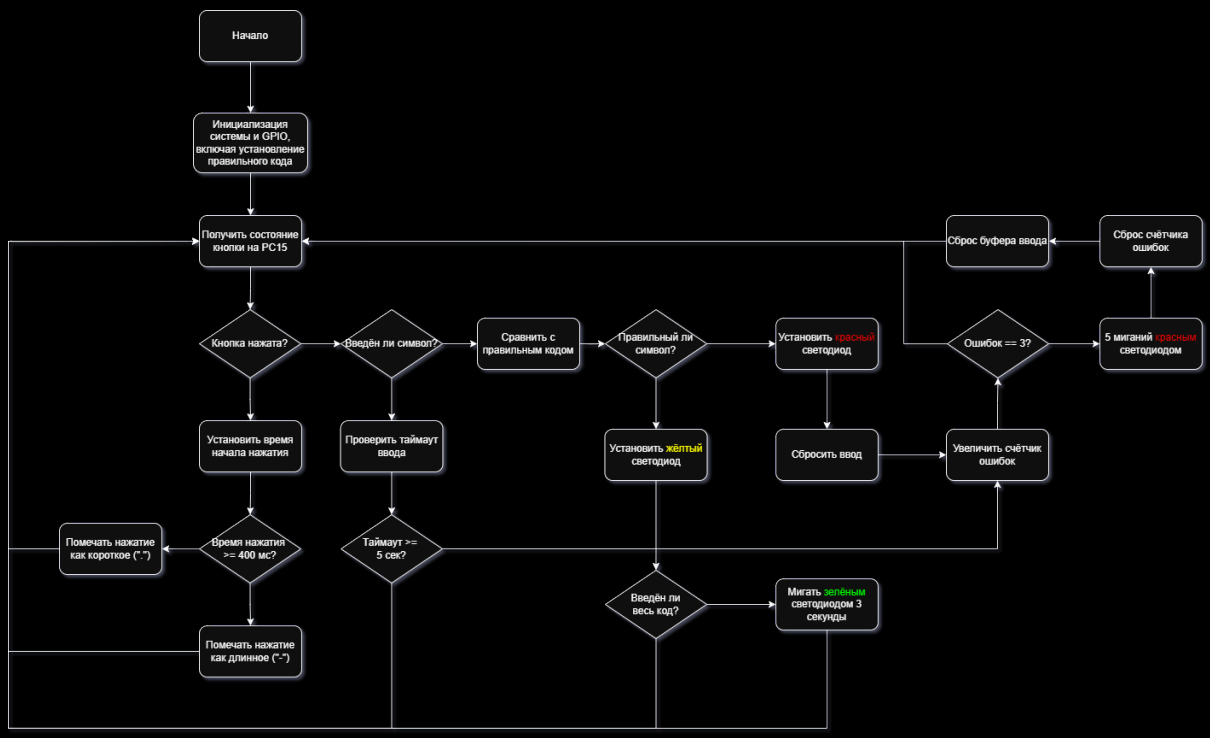
2.1. Используемые контакты



Описание контактов:

- PC15 — перехватывает нажатие кнопки (настроен на GPIO_Input)
- PD13 — управляет зеленым светодиодом (настроен на GPIO_Output)
- PD14 — управляет жёлтым светодиодом (настроен на GPIO_Output)
- PD15 — управляет красным светодиодом (настроен на GPIO_Output)
- PB3, PB4, PA13, PA14, PA15 — JTag для отладки

2.2. Блок-схема:



2.3. Код драйвера

```
#include "main.h"
#include <string.h>
#include <stdbool.h>

/* Private function prototypes
-----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
void Error_Handler(void);

/* USER CODE BEGIN 0 */
/******************* HARDWARE DRIVERS (universal, non-blocking)
*******************/
// LED Driver - hardware abstraction layer
static void LED_SetGreen(bool on) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, on ? GPIO_PIN_SET :
GPIO_PIN_RESET);
}

static void LED_SetYellow(bool on) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, on ? GPIO_PIN_SET :
GPIO_PIN_RESET);
}

static void LED_SetRed(bool on) {
```

```
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, on ? GPIO_PIN_SET :
GPIO_PIN_RESET);
}

// Button Driver with debouncing (non-blocking)
#define DEBOUNCE_DELAY_MS 40
static bool button_debounced_state = true;  // true = released
(active-low)
static uint32_t last_debounce_time = 0;
static bool button_pressed_event = false;
static bool button_released_event = false;
static uint32_t press_start_time = 0;
static uint32_t last_press_duration = 0;

void Button_Process(void) {
    uint32_t now = HAL_GetTick();
    bool raw_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15) ==
GPIO_PIN_RESET; // Active-low

    if (raw_state != button_debounced_state) {
        if (now - last_debounce_time > DEBOUNCE_DELAY_MS) {
            button_debounced_state = raw_state;
            last_debounce_time = now;

            if (button_debounced_state) { // Pressed event
                button_pressed_event = true;
                press_start_time = now;
            } else { // Released event
                button_released_event = true;
                last_press_duration = now - press_start_time;
            }
        }
    }
}

bool Button_WasPressed(void) {
    bool state = button_pressed_event;
    button_pressed_event = false;
    return state;
}

bool Button_WasReleased(void) {
    bool state = button_released_event;
    button_released_event = false;
    return state;
}
```

```
uint32_t Button_GetPressDuration(void) {
    return last_press_duration;
}

/******************* APPLICATION LOGIC *******************/
// Morse code parameters
const char correct_code[] = ".-.-.-.."; // 8 symbols
const int code_length = 8;
const uint32_t dot_threshold_ms = 400;
const uint32_t input_timeout_ms = 5000;
const uint32_t yellow_blink_ms = 150;
const uint32_t red_blink_ms = 300;
const uint32_t unlock_duration_ms = 3000;
const int max_failures = 3;

// System states
typedef enum {
    STATE_WAITING,           // Waiting for first button press
    STATE_INPUT,             // Actively reading code sequence
    STATE_YELLOW_BLINK_ON,   // Yellow LED turned on (waiting for
duration)
    STATE_YELLOW_BLINK_OFF,  // Yellow LED turned off (processing
complete)
    STATE_RED_BLINK_ON,      // Red LED turned on (waiting for
duration)
    STATE_RED_BLINK_OFF,     // Red LED turned off (processing
complete)
    STATE_UNLOCKED,          // Green LED on for 3 seconds
    STATE_FAILURE_SEQUENCE   // 5 red blinks after 3 failures
} SystemState;

SystemState current_state = STATE_WAITING;
int input_index = 0;         // Tracks position in code sequence
int failure_count = 0;       // Counts consecutive failures
uint32_t last_input_time = 0;
uint32_t blink_start_time = 0;
int blink_count = 0;         // For failure blink sequence

// Reset all state variables to initial conditions
void Reset_State(void) {
    input_index = 0;
    failure_count = 0;
    blink_count = 0;
    LED_SetGreen(false);
    LED_SetYellow(false);
```

```
    LED_SetRed(false);
}

// Reset only input buffer (keep failure count for lockout)
void Reset_InputBuffer(void) {
    input_index = 0;
    LED_SetYellow(false);
    LED_SetRed(false);
}

/* USER CODE END 0 */

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    // Initialize all LEDs to off
    Reset_State();

    while (1) {
        uint32_t now = HAL_GetTick();

        // Process button with debouncing
        Button_Process();

        switch(current_state) {
            case STATE_WAITING:
            {
                LED_SetGreen(false);
                LED_SetYellow(false);
                LED_SetRed(false);

                if (Button_WasPressed()) {
                    last_input_time = now;
                    current_state = STATE_INPUT;
                }
                break;
            }

            case STATE_INPUT:
            {
                // Handle input timeout - reset if no activity for
5 seconds
                if (input_index > 0 && (now - last_input_time >=
input_timeout_ms)) {
```

```
                    LED_SetYellow(true);
                    blink_start_time = now;
                    current_state = STATE_YELLOW_BLINK_ON;
                    // Special case: this is a timeout indication
                    input_index = -1; // Use negative index to
indicate timeout
                    break;
                }

                if (Button_WasReleased()) {
                    last_input_time = now;
                    uint32_t press_duration =
Button_GetPressDuration();
                    char symbol = (press_duration <
dot_threshold_ms) ? '.' : '-';

                    if (input_index < code_length) {
                        if (symbol == correct_code[input_index]) {
                            // CORRECT SYMBOL
                            LED_SetYellow(true);
                            blink_start_time = now;
                            current_state = STATE_YELLOW_BLINK_ON;
                        } else {
                            // WRONG SYMBOL
                            LED_SetRed(true);
                            blink_start_time = now;
                            current_state = STATE_RED_BLINK_ON;
                        }
                    }
                }
                break;
            }

            case STATE_YELLOW_BLINK_ON:
            {
                // Wait for blink duration to pass
                if (now - blink_start_time >= yellow_blink_ms) {
                    LED_SetYellow(false);
                    current_state = STATE_YELLOW_BLINK_OFF;
                }
                break;
            }

            case STATE_YELLOW_BLINK_OFF:
            {
                // Process what happens after yellow blink
```

```
                if (input_index == -1) {
                    // This was a timeout indication
                    Reset_InputBuffer();
                    current_state = STATE_WAITING;
                } else {
                    input_index++; // Move to next symbol

                    if (input_index >= code_length) {
                        // FULL CODE CORRECT - unlock
                        LED_SetGreen(true);
                        blink_start_time = now;
                        current_state = STATE_UNLOCKED;
                        failure_count = 0; // Reset failures on
success
                    } else {
                        // Continue to next symbol
                        current_state = STATE_INPUT;
                    }
                }
                break;
            }

            case STATE_RED_BLINK_ON:
            {
                // Wait for blink duration to pass
                if (now - blink_start_time >= red_blink_ms) {
                    LED_SetRed(false);
                    current_state = STATE_RED_BLINK_OFF;
                }
                break;
            }

            case STATE_RED_BLINK_OFF:
            {
                // Process what happens after red blink
                failure_count++;
                Reset_InputBuffer(); // Reset input buffer after
wrong symbol

                if (failure_count >= max_failures) {
                    // Start 5-blink failure sequence
                    blink_count = 0;
                    LED_SetRed(true);
                    blink_start_time = now;
                    current_state = STATE_FAILURE_SEQUENCE;
                    failure_count = 0; // Reset for next cycle
```

```
                } else {
                    current_state = STATE_WAITING;
                }
                break;
            }

            case STATE_UNLOCKED:
            {
                if (now - blink_start_time >= unlock_duration_ms) {
                    LED_SetGreen(false);
                    Reset_State(); // Complete reset after
successful unlock
                    current_state = STATE_WAITING;
                }
                // Clear any button events during unlock period
                Button_WasPressed();
                Button_WasReleased();
                break;
            }

            case STATE_FAILURE_SEQUENCE:
            {
                uint32_t elapsed = now - blink_start_time;

                if (blink_count % 2 == 0) { // ON period (300ms)
                    if (elapsed >= 300) {
                        LED_SetRed(false);
                        blink_count++;
                        blink_start_time = now;
                    }
                } else { // OFF period (200ms)
                    if (elapsed >= 200) {
                        blink_count++;

                        if (blink_count >= 10) { // 5 full blinks =
10 state changes
                            LED_SetRed(false);
                            Reset_State(); // COMPLETE RESET AFTER
FAILURE SEQUENCE
                            current_state = STATE_WAITING;
                        } else {
                            LED_SetRed(true);
                            blink_start_time = now;
                        }
                    }
                }
```

```
                // Clear button events during blink sequence
                Button_WasPressed();
                Button_WasReleased();
                break;
            }
        }

        // Short delay to prevent CPU hogging
        for(volatile int i = 0; i < 10; i++) {
            __NOP();
        }
    }
}
```

3. Вывод

В ходе выполнения работы были освоены принципы работы с портами ввода-вывода общего назначения (GPIO). На практике эти знания были применены для разработки и программирования кодового замка, использующего трёх светодиодов в качестве индикатора состояния.